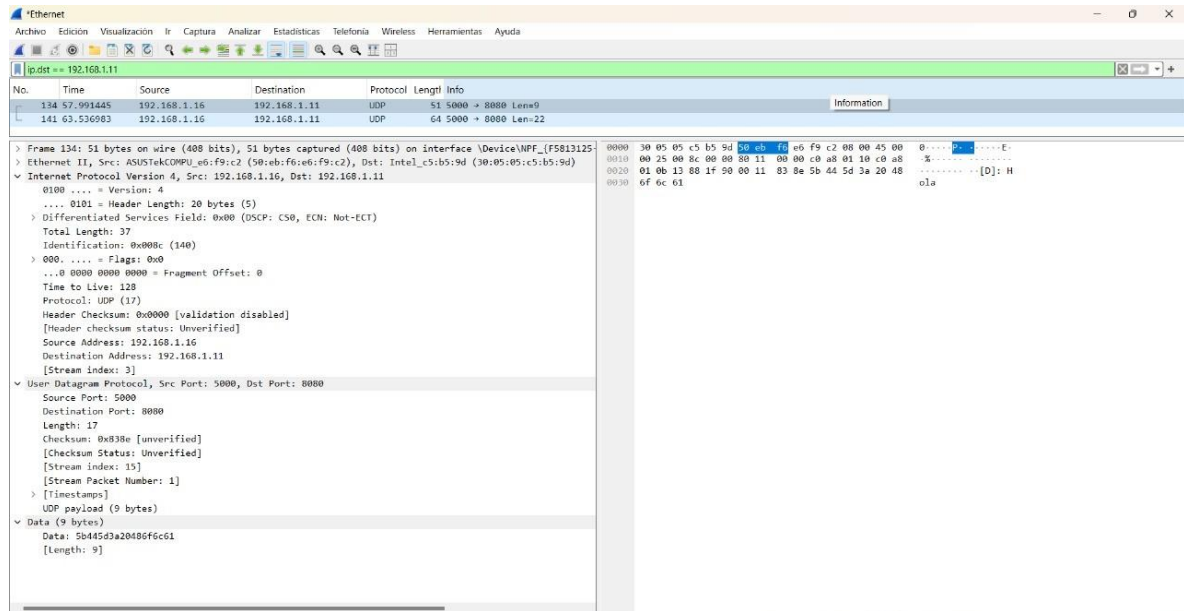


TALLER UDP CONNECTION #1 – COMPUTACIÓN EN INTERNET I

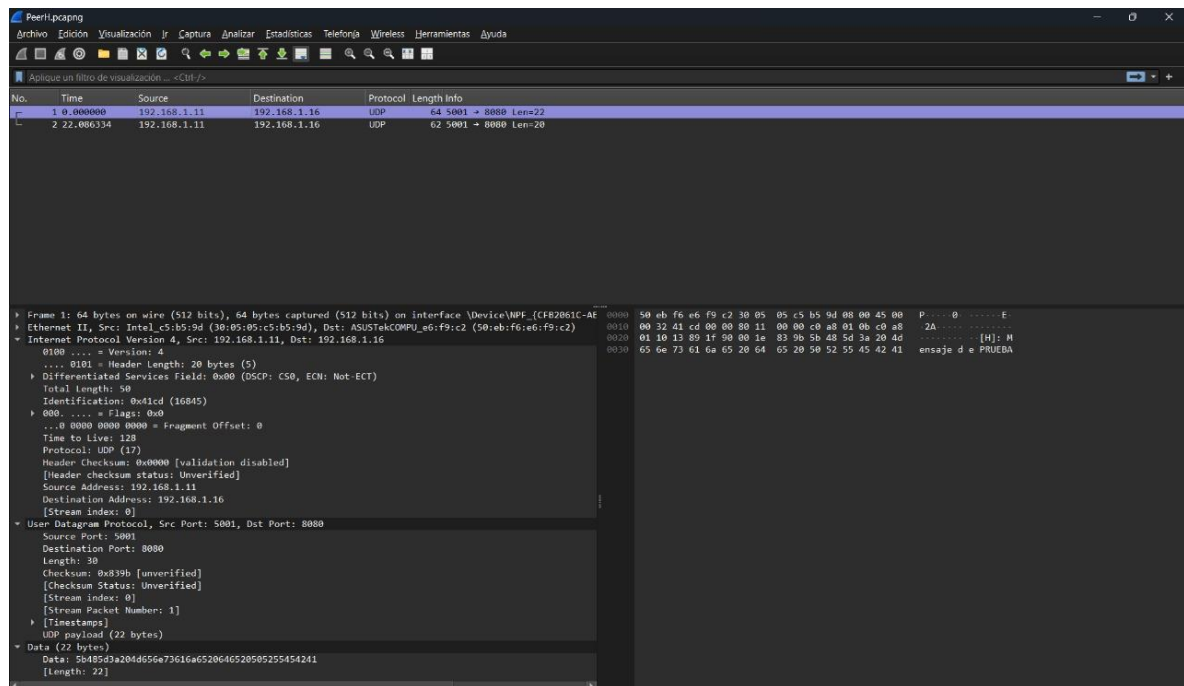
INTEGRANTES: Daniel Esteban Arcos Cerón [A00400760]

Hideki Tamura Hernández [A00348618]

0. Capturas de Wireshark para seguimiento de paquetes enviados por cada Peer:



2 Captura de Wireshark desde Peer D



1 Captura de Wireshark desde Peer H

Nota: En este caso, los **peers** son **D** y **H**, en lugar de **A** y **B**.

1. ¿Es posible ver en la captura de Wireshark el contenido del mensaje enviado? [10%]

R/ Sí, en la captura de Wireshark es posible observar el contenido de los mensajes enviados, ya que los mensajes UDP en la implementación actual se envían en texto plano sin ningún tipo de cifrado. Esto significa que Wireshark puede interceptar y mostrar el texto exacto que se transmite en cada paquete. En general, al no haberse aplicado ningún mecanismo de cifrado ni compresión en los datos transmitidos, se permite la inspección directa del contenido.

2. ¿Cuál es el *checksum* de la captura? ¿Explique/investiguen porque este checksum? [10%]

R/ Primera captura (desde PeerD):

- **Checksum del encabezado IP:** 0x0000 [validation disabled]
- **Checksum del encabezado UDP:** 0x838e [unverified]

Segunda captura (desde PeerH):

- **Checksum del encabezado IP:** 0x0000 [validation disabled]
- **Checksum del encabezado UDP:** 0x839b [unverified]

Análisis en Wireshark: Las capturas muestran que la validación de checksums para los encabezados IP y UDP está desactivada (validation disabled). Esto ocurre porque muchas tarjetas de red modernas realizan el cálculo y la verificación de checksums por hardware (offloading), y el software de captura (Wireshark) no recalcula ni verifica estos valores, mostrando el estado como "unverified" o ceros. Lo anterior explica por qué en las capturas aparece el checksum IP en cero o el checksum UDP sin validar.

Checksum en UDP [Respaldo teórico]: Como sabemos, **UDP** es un protocolo que utiliza un **checksum** para verificar la integridad de los datos transmitidos. En **IPv4**, el uso del checksum es opcional, mientras que en **IPv6** es obligatorio. A diferencia de **TCP**, UDP no garantiza la entrega ni la corrección de errores; su checksum se emplea únicamente para detectar posibles errores en los datos recibidos. Por ende, la validación y cálculo de este checksum pueden depender de la configuración del sistema y de si la tarjeta de red realiza el procesamiento por hardware (offloading).

3. ¿Qué patrones de diseño / arquitectura aplicaría al desarrollo de un programa basado en red como este? [15%] (Lista simple de propuestas)

R/ A continuación, presentamos algunos patrones de diseño y arquitecturas de software que, en nuestra opinión, resultan adecuados para el desarrollo de aplicaciones de red, como el sistema de mensajería por UDP implementado en este proyecto:

- **PATRONES DE DISEÑO:**

- **Observer** → Es un patrón de comportamiento que define un mecanismo de suscripción mediante el cual múltiples objetos pueden recibir notificaciones sobre cualquier evento que ocurra en el objeto que están observando. En aplicaciones de red como esta, resultaría útil para gestionar eventos como la recepción de mensajes. Por ejemplo, al recibir un paquete, el sistema puede notificar automáticamente a los módulos correspondientes, como la interfaz gráfica, sin necesidad de acoplar directamente la lógica de red con la capa de presentación.
- **Strategy** → Es un patrón de comportamiento que permite definir una familia de algoritmos, encapsular cada uno en una clase independiente y hacerlos intercambiables en tiempo de ejecución. En una aplicación P2P, puede utilizarse para adaptar de forma dinámica estrategias como los reintentos de envío, el cifrado de mensajes o la gestión de errores, sin necesidad de modificar la lógica principal del cliente.

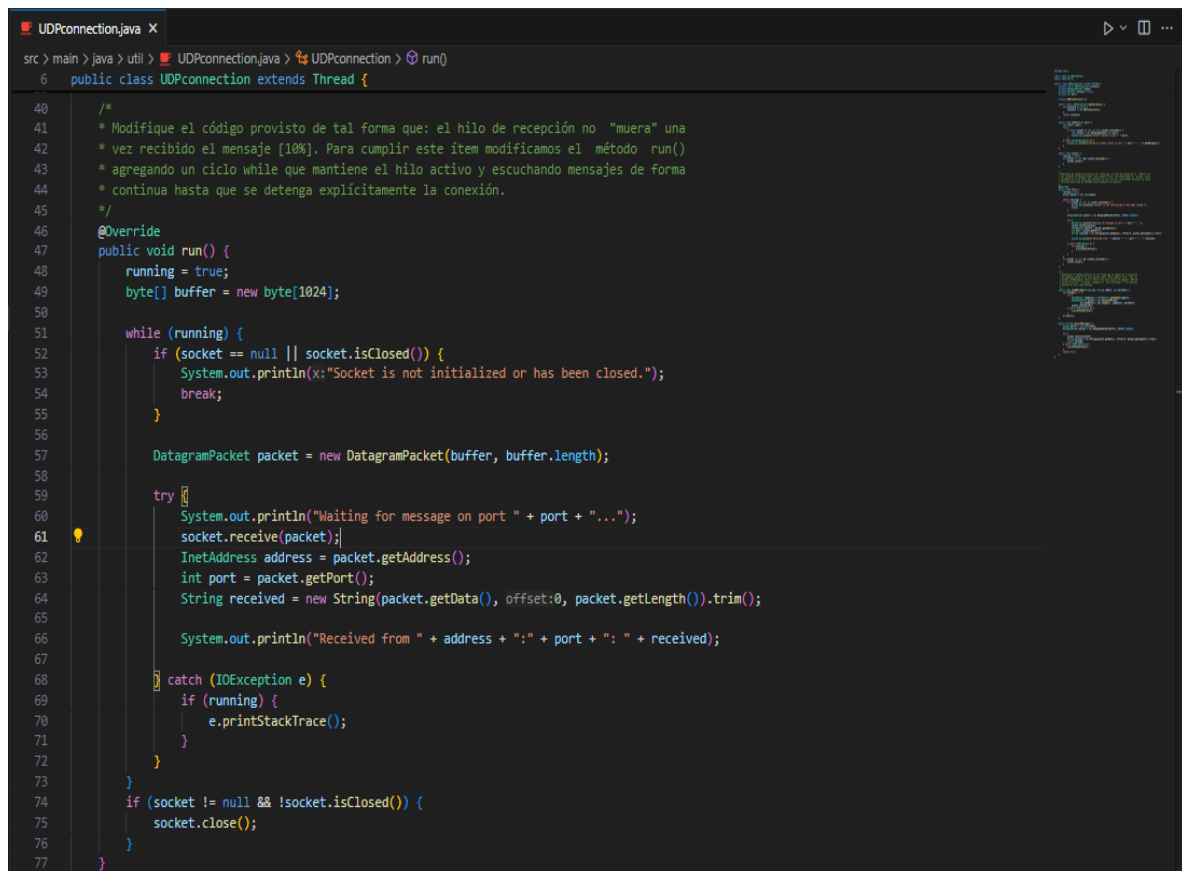
- **ARQUITECTURAS:**

- **Arquitectura en Capas (Layered Architecture)** → Este enfoque divide el sistema en capas bien definidas, cada una con responsabilidades específicas, como presentación, lógica de negocio y acceso a red. En el contexto de este proyecto, permitiría organizar el código en módulos como la interfaz de usuario, la lógica que controla el flujo de mensajes, los servicios que procesan o validan los datos, y la capa de red encargada del envío y recepción de paquetes UDP.
- **Cliente-Servidor** → Aunque el sistema actual funciona bajo un modelo peer-to-peer, adoptar una arquitectura cliente-servidor podría ser útil en futuras versiones (aunque no aplicaría en esta etapa). Esta estructura permitiría una gestión centralizada de sesiones, usuarios y almacenamiento de mensajes, lo cual sería beneficioso si se busca mayor control, persistencia de datos o administración desde un único punto.

4. Modifique el código provisto de tal forma que: el hilo de recepción no “muera” una vez recibido el mensaje. [10%]

R/ En el código original, el hilo de recepción (**método run()**) de la clase **UDPconnection** finalizaba después de recibir un solo mensaje debido a su implementación inicial. Sin embargo, en la versión actual del proyecto, este comportamiento fue modificado para que el hilo de recepción permanezca activo continuamente, esperando y procesando mensajes de manera indefinida hasta que se le indique que se detenga.

La clave para lograr esto es el uso de un bucle **while (running)** dentro del método **run()**. Este bucle mantiene el hilo en ejecución, recibiendo mensajes constantemente, lo que permite que la aplicación escuche múltiples mensajes **UDP** durante su ciclo de vida:



```
UDPconnection.java x
src > main > java > util > UDPconnection.java > UDPconnection > run()
6 public class UDPconnection extends Thread {
40     /*
41     * Modifique el código provisto de tal forma que: el hilo de recepción no "muera" una
42     * vez recibido el mensaje [10%]. Para cumplir este ítem modificamos el método run()
43     * agregando un ciclo while que mantiene el hilo activo y escuchando mensajes de forma
44     * continua hasta que se detenga explícitamente la conexión.
45     */
46     @Override
47     public void run() {
48         running = true;
49         byte[] buffer = new byte[1024];
50
51         while (running) {
52             if (socket == null || socket.isClosed()) {
53                 System.out.println(x:"Socket is not initialized or has been closed.");
54                 break;
55             }
56
57             DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
58
59             try {
60                 System.out.println("Waiting for message on port " + port + "...");
61                 socket.receive(packet);
62                 InetAddress address = packet.getAddress();
63                 int port = packet.getPort();
64                 String received = new String(packet.getData(), 0, packet.getLength()).trim();
65
66                 System.out.println("Received from " + address + ":" + port + ": " + received);
67
68             } catch (IOException e) {
69                 if (running) {
70                     e.printStackTrace();
71                 }
72             }
73
74             if (socket != null && !socket.isClosed()) {
75                 socket.close();
76             }
77         }
78     }
79 }
```

5. Modifique el código provisto de tal forma que la lógica de transmisión de paquetes quede en un hilo aparte. [10%]

R/ Para cumplir este ítem, se modificó el método **sendMessage()** de la clase **UDPconnection** para que la lógica de envío de paquetes **UDP** se ejecute en un hilo separado. Esto se logra creando un nuevo hilo cada vez que se envía un mensaje, encapsulando la lógica de construcción y envío del **DatagramPacket** dentro de un **Runnable** que se ejecuta asíncronamente.

Esta implementación evita que el **hilo principal** o **la interfaz gráfica** se bloqueen mientras el mensaje se transmite, mejorando la capacidad de respuesta y la experiencia del usuario. Además, permite que la aplicación continúe ejecutándose y procesando otras tareas sin esperar a que la operación de red termine.

```
UDPConnection.java x
src > main > java > util > UDPConnection.java > UDPConnection > run()
6 public class UDPConnection extends Thread {

79 /**
80  * Modifique el código provisto de tal forma que la lógica de transmisión
81  * de paquetes quede en un hilo aparte [10%]. Para cumplir este ítem, el
82  * método sendMessage() crea un nuevo hilo para enviar el paquete UDP de
83  * forma asíncrona, evitando bloquear el hilo principal o la interfaz
84  * durante el envío del mensaje.
85  */
86 public void sendMessage(String msj, String ipDest, int portDest) {
87     new Thread() -> {
88         try {
89             InetAddress ipAddress = InetAddress.getByName(ipDest);
90             DatagramPacket packet = new DatagramPacket(
91                 msj.getBytes(), msj.length(), ipAddress, portDest);
92             socket.send(packet);
93         } catch (IOException e) {
94             e.printStackTrace();
95         }
96     }).start();
97 }

98
99 public String receiveMessage() {
100     byte[] buffer = new byte[1024];
101     DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
102
103     try {
104         socket.receive(packet);
105         String message = new String(packet.getData(), 0, packet.getLength()).trim();
106         return message;
107     } catch (IOException e) {
108         e.printStackTrace();
109     }
110     return null;
111 }
```

6. Investiguen que modificaciones son necesarias para implementar este mismo sistema, pero para la comunicación TCP en java. [10%]

R/ Para adaptar este sistema a una comunicación basada en **TCP en Java**, es necesario realizar varias modificaciones clave. En primer lugar, se debe reemplazar el uso de **DatagramSocket** por un **ServerSocket** para la parte que escucha conexiones entrantes, y un **Socket** para la comunicación bidireccional con cada cliente o peer.

Además, en lugar de usar **DatagramPacket** para enviar y recibir mensajes, se debe utilizar flujos de datos continuos mediante **InputStream** y **OutputStream** asociados al **socket**, ya que **TCP** es un protocolo orientado a conexión basado en flujos y no en paquetes. En este sentido, también es fundamental gestionar correctamente el ciclo de vida de las conexiones **TCP**: establecer la conexión, mantenerla activa durante la comunicación y cerrarla adecuadamente cuando sea necesario.

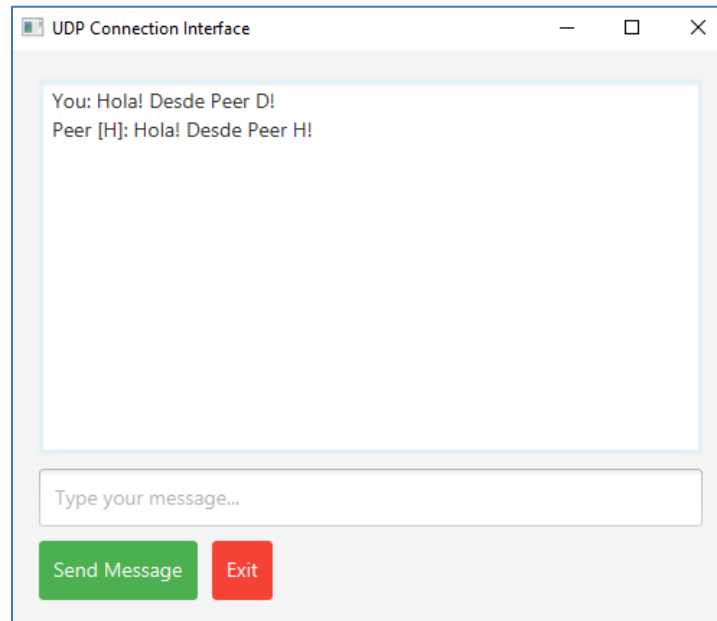
Finalmente, al trabajar con **TCP** se **elimina la lógica de empaquetado y desempaquetado** manual de mensajes, ya que la transmisión en este protocolo es un flujo continuo de bytes garantizado en orden y sin pérdidas, a diferencia de **UDP**.

7. ¿Qué utilidades de codificación o seguridad agregaría al código? [10%]

R/ Para reforzar la seguridad en el sistema de comunicación, se pueden integrar diversas utilidades y técnicas de seguridad propias de Java. A continuación, presentamos algunas de las soluciones más destacadas que podrían implementarse en el código:

- **Cifrado de mensajes:** El uso de algoritmos de cifrado como **AES (Advanced Encryption Standard)** puede asegurar que los mensajes enviados sean legibles solo por los destinatarios autorizados. El cifrado de extremo a extremo asegura que, incluso si los datos son interceptados en tránsito, no podrán ser comprendidos sin la clave adecuada.
- **Autenticación de mensajes:** Para verificar la autenticidad de los mensajes y asegurarse de que provienen de una fuente confiable, se puede utilizar **HMAC (Hash-based Message Authentication Code)**. Este método combina un algoritmo de hash seguro (como **SHA-256**) con una clave secreta para generar un código de autenticación. De este modo, el receptor puede verificar que el mensaje no ha sido alterado durante su transmisión.
- **Verificación de integridad:** El uso de **CRC32** o **SHA-256** para verificar la integridad de los datos transmitidos permite detectar cualquier alteración no deseada en los mensajes. Aunque **CRC32** es adecuado para detectar errores simples, **SHA-256** es más robusto y proporciona una mayor seguridad para garantizar que los datos no hayan sido manipulados.

8. Bonus: desarrollen una interfaz de usuario en JavaFX para este programa. [15%]



R/ Para implementar y ejecutar correctamente la interfaz de usuario en **JavaFX** se realizaron varios ajustes importantes en el proyecto. Inicialmente, el proyecto no estaba preparado para integrar **JavaFX** con **Maven**, por lo que fue necesario reestructurar la organización de las carpetas y actualizar el archivo **pom.xml** para incluir las dependencias oficiales de **JavaFX**:

```

pom.xml X
pom.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6
7   <groupId>com.example</groupId>
8   <artifactId>udp-connection-dh</artifactId>
9   <version>1.0-SNAPSHOT</version>
10
11   <properties>
12     <maven.compiler.source>11</maven.compiler.source>
13     <maven.compiler.target>11</maven.compiler.target>
14     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15   </properties>
16
17   <dependencies>
18     <dependency>
19       <groupId>org.openjfx</groupId>
20       <artifactId>javafx-controls</artifactId>
21       <version>17.0.6</version>
22     </dependency>
23     <dependency>
24       <groupId>org.openjfx</groupId>
25       <artifactId>javafx-fxml</artifactId>
26       <version>17.0.6</version>
27     </dependency>
28     <dependency>
29       <groupId>org.openjfx</groupId>
30       <artifactId>javafx-graphics</artifactId>
31       <version>17.0.6</version>
32     </dependency>
33   </dependencies>
34

```

Por otro lado, se creó la clase principal **Chat**, ubicada en el paquete **ui**, la cual extiende la clase **Application** de **JavaFX** y funciona como el punto de entrada de la interfaz gráfica. Esta clase contiene un área de texto (**TextArea**) para mostrar el historial de mensajes enviados y recibidos, así como un campo de texto (**TextField**) para que el usuario pueda ingresar nuevos mensajes. Además, incluye botones para enviar mensajes y para salir de la aplicación.

```

Chat.java X
src > main > java > ui > Chat.java > Chat > start(stage)
10
11 public class Chat extends Application {
12
13     private int localPort;
14     private int remotePort;
15     private String remoteIP;
16     private UDPConnection udpConnection;
17     private volatile boolean running = true;
18
19     @Override
20     public void start(Stage primaryStage) {
21         // Área para mostrar los mensajes .....
22         TextArea chatArea = new TextArea();
23         chatArea.setEditable(false);
24         chatArea.setPrefSize(400, 300);
25         chatArea.setStyle(value: "-fx-background-color: #f0f0f0; -fx-font-size: 14px;");
26
27         // Campo para escribir los mensajes .....
28         TextField inputField = new TextField();
29         inputField.setPromptText("Type your message...");
30         inputField.setStyle(value: "-fx-padding: 10px; -fx-font-size: 14px;");
31
32         Button sendButton = new Button(text: "Send Message");
33         sendButton.setStyle(value: "-fx-background-color: #4CAF50; -fx-text-fill: white; -fx-font-size: 14px; -fx-padding: 10px;");
34
35         // Recordatorio para usuario: Revisar PeerConfig.java!!!!!!!!!!!!
36         localPort = PeerConfig.PORT_PEER_D; // Puerto local para escuchar
37
38         remoteIP = PeerConfig.IP_PEER_H; // IP destino para enviar
39         remotePort = PeerConfig.PORT_PEER_H; // Puerto destino para enviar
40
41         // Enviar mensaje cuando se presiona el botón.....
42         sendButton.setOnAction(e -> {
43             String message = inputField.getText();
44             if (!message.isEmpty()) {
45                 udpConnection.sendMessage(message, remoteIP, remotePort); // Ajustar según caso en PeerConfig!
46                 chatArea.appendText("You: " + message + "\n"); // Mostrar el mensaje enviado en la interfaz!
47                 inputField.clear(); // Limpiar el campo de texto!
48             }
49         });
50     }
51 }

```

La integración con la lógica de comunicación **UDP** se realiza configurando la conexión con los puertos y direcciones IP definidos en la clase **PeerConfig**. Para la recepción de mensajes, se crea un hilo dedicado que permanece escuchando de manera continua en el **socket UDP**, y que actualiza el área de texto de la interfaz usando el método **Platform.runLater()**, garantizando la correcta sincronización con el hilo principal. El envío de mensajes se maneja a través del **botón de envío**, que utiliza la clase **UDPconnection** para transmitir mensajes de forma asíncrona sin bloquear la interfaz.

```
PeerConfig.java X
src > main > java > ui > PeerConfig.java > {} ui
1 package ui;
2
3 public final class PeerConfig {
4     /*
5      * Clase centralizada para configurar las direcciones IP y puertos
6      * de cada peer involucrado en la comunicación UDP. Aquí debe hacerse
7      * la configuración para la comunicación: definir las IPs y puertos
8      * locales y remotos para cada nodo.
9      */
10    public static final String IP_PEER_D = "127.0.0.1";
11    public static final int PORT_PEER_D = 5000;
12
13    public static final String IP_PEER_H = "127.0.0.1";
14    public static final int PORT_PEER_H = 5001;
15
16    private PeerConfig() {}
17 }
```

```
// Configurar la conexión UDP:
udpConnection = UDPconnection.getInstance();
udpConnection.setPort(localPort); // Puerto local para escuchar. Por default es el de Peer D!
udpConnection.start();

// Recibir mensajes en un hilo separado y actualizar el chat:
Thread receiverThread = new Thread(() -> {
    while (running) {
        String receivedMessage = udpConnection.receiveMessage();
        if (receivedMessage != null) {
            Platform.runLater(() -> chatArea.appendText("Peer " + receivedMessage + "\n"));
        }
    }
});
receiverThread.setDaemon(true);
receiverThread.start();
```

Finalmente, se implementó un manejo adecuado del ciclo de vida de la aplicación, con un método **shutdown()** que se encarga de cerrar la conexión **UDP** y detener los hilos activos cuando la aplicación se cierra. El método **stop()** de **JavaFX** es sobrescrito para asegurar que esta limpieza se realice siempre que la aplicación finalice.

```
private void shutdown() {
    running = false;
    if (udpConnection != null) {
        udpConnection.close();
    }
}

@Override
public void stop() throws Exception {
    shutdown();
    super.stop();
}
```